

API Integration Documentation

REST APIs, API Authentication, API Calling Methods, JSON Handling, and Third-Party API Integration

Prepared for interview-ready understanding and practical project implementation

Prepared by	Parth Goyal
Topic	API Integration
Focus Area	Modern Web Applications and Backend Connectivity
Purpose	Submission and Revision Notes

Core Idea

API integration connects different applications or services so they can exchange data securely and reliably.

1. Introduction to API Integration

API integration is a core part of modern software applications. Most applications do not work alone. They connect with backend servers, databases, payment gateways, authentication providers, AI services, maps, email services, and other external systems. This connection usually happens through APIs.

In simple words, API integration means connecting two applications or services through APIs so that they can send and receive data. For example, a frontend website can call a FastAPI backend to log in a user, fetch products, create orders, or get a machine learning prediction.

Example	How API Integration Works
Login system	Frontend sends email and password to backend login API. Backend validates credentials and returns a token.
Weather app	Application calls a weather API to get live temperature and weather details.
Payment system	Backend integrates with Razorpay or Stripe to process payments securely.
AI chatbot	Application sends a prompt to an AI API and receives a generated response.
Streamlit + FastAPI	Streamlit collects user input and calls FastAPI for prediction or database logic.

Interview-ready definition: API integration is the process of connecting different applications or services through APIs so they can exchange data. It is used for frontend-backend communication, third-party services, authentication, payments, maps, AI APIs, and many other modern application features.

2. Basic API Integration Flow

A common API integration flow starts from the frontend. The user performs an action, such as clicking a login button or submitting a form. JavaScript collects the input data and sends an API request to the backend. The backend validates and processes the request, communicates with the database or another external service if required, and returns a response to the frontend.

```
User action on frontend
↓
JavaScript sends API request
↓
Backend receives request
↓
Request validation and business logic
↓
Database or third-party API interaction
↓
Backend returns JSON response
↓
Frontend updates UI
```

Example: In a login system, the frontend sends a POST request to the /login API with email and password. The backend checks the user from the database, verifies the password, and returns either a success response with token or an error response.

3. API Request and API Response

3.1 API Request

An API request is the message sent by the client to the server. It tells the server what operation is required. A request may include a URL, HTTP method, headers, path parameters, query parameters, and request body.

Part of Request	Meaning	Example
URL	The address of the API endpoint	http://localhost:8000/users
HTTP Method	Operation to perform	GET, POST, PUT, PATCH, DELETE

Part of Request	Meaning	Example
Headers	Extra request information	Content-Type: application/json
Path Parameter	Dynamic value inside URL path	/users/10
Query Parameter	Filtering or searching values	/products?category=mobile
Request Body	JSON data sent to server	{"name": "Parth"}

3.2 API Response

An API response is the answer sent by the server after processing the request. It usually contains a status code, response body, headers, and message.

```
{
  "message": "User created successfully",
  "user": {
    "id": 1,
    "name": "Parth"
  }
}
```

Status Code	Meaning	Common Use
200	OK	Request successful
201	Created	New resource created successfully
400	Bad Request	Invalid input or business rule failure
401	Unauthorized	Login/token missing or invalid
403	Forbidden	User is authenticated but has no permission
404	Not Found	Resource not found
422	Validation Error	Request does not match expected schema
500	Internal Server Error	Unexpected server-side error

4. REST APIs

REST API is one of the most commonly used API styles. REST APIs use resource-based URLs and HTTP methods to perform operations. In a REST API, the URL usually represents the resource, and the HTTP method represents the action.

HTTP Method	Purpose	Example
GET	Fetch or read data	GET /users
POST	Create new data or send data	POST /users
PUT	Full update or replace a resource	PUT /users/1
PATCH	Partial update of selected fields	PATCH /users/1
DELETE	Delete a resource	DELETE /users/1

A good REST API avoids writing actions directly in the URL. Instead of using /getUser or /deleteUser, we use resource-based endpoints like GET /users and DELETE /users/1.

```
Good REST style:
GET /users      → Fetch all users
GET /users/1   → Fetch user with ID 1
POST /users    → Create a new user
PATCH /users/1 → Update selected fields of user 1
DELETE /users/1 → Delete user 1
```

5. JSON Handling

JSON stands for JavaScript Object Notation. It is the most common data format used in API integration because it is lightweight, readable, and easy for both frontend and backend to process.

```
{
  "id": 1,
  "name": "Parth",
  "email": "parth@gmail.com",
  "skills": ["Python", "FastAPI", "JavaScript"],
  "is_active": true
}
```

5.1 JSON.stringify()

When we send a JavaScript object to the backend in a POST or PATCH request, we convert it into a JSON string using `JSON.stringify()`.

```
const user = {
  name: "Parth",
  email: "parth@gmail.com"
};

const jsonData = JSON.stringify(user);
```

5.2 response.json()

When the backend sends JSON data to the frontend, we convert the response into a JavaScript object using `response.json()`.

```
const response = await fetch("http://localhost:8000/users");
const data = await response.json();
console.log(data);
```

Important line: `JSON.stringify()` converts a JavaScript object into a JSON string before sending it to the backend, while `response.json()` converts the backend JSON response into a JavaScript object.

6. API Calling Methods

APIs can be called from different tools and programming environments. In frontend JavaScript, `fetch` and `Axios` are common. In Python applications such as `Streamlit`, the `requests` library is common. For testing, `Postman`, `Swagger UI`, and `cURL` are commonly used.

Method/Tool	Where Used	Purpose
<code>fetch</code>	Frontend JavaScript	Built-in browser API for HTTP requests
<code>Axios</code>	Frontend JavaScript/React	External library for easier API calls
<code>requests</code>	Python/Streamlit	Call APIs from Python code
<code>Postman</code>	Testing	Manual API testing
<code>Swagger UI</code>	FastAPI docs	Test APIs directly from browser
<code>cURL</code>	Terminal	Command-line API testing

7. API Calls Using fetch()

`fetch()` is a built-in browser function used to make API calls from JavaScript. It returns a Promise, so it is commonly used with `async-await` for cleaner code.

7.1 GET Request

```
async function getUsers() {
  try {
    const response = await fetch("http://localhost:8000/users");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("Network Error:", error);
  }
}

getUsers();
```

7.2 POST Request

```
async function createUser() {
  try {
    const response = await fetch("http://localhost:8000/users", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        name: "Parth",
        email: "parth@gmail.com",
        password: "12345"
      })
    });

    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("Network Error:", error);
  }
}

createUser();
```

In a POST request, method tells the backend what operation we want to perform, headers tell the backend that the data is JSON, and body contains the actual data after converting it with `JSON.stringify()`.

8. Asynchronous API Calls and Error Handling

API calls are asynchronous because the server may take time to respond. JavaScript uses promises, `async`, and `await` to handle this. The `async` keyword makes a function asynchronous, and `await` waits for a promise to complete.

```
async function getUser() {
  const response = await fetch("http://localhost:8000/users/1");
  const data = await response.json();
  console.log(data);
}
```

Error handling is important in API integration. There are usually two types of errors: API response errors and network errors. API response errors happen when the server returns a status like 404 or 401. Network errors happen when the server is unreachable or the URL is wrong.

```
async function getUser() {
  try {
    const response = await fetch("http://localhost:8000/users/99");
    const data = await response.json();

    if (!response.ok) {
      console.log("API Error:", data.detail);
    }
  }
}
```

```

    return;
  }

  console.log("User:", data);
} catch (error) {
  console.log("Network Error:", error);
}
}
}

```

9. Axios for API Calls

Axios is an external JavaScript library used for making HTTP requests. It is commonly used in React and other frontend projects because it simplifies API calls and automatically converts JSON responses.

```

npm install axios
import axios from "axios";

async function getUsers() {
  try {
    const response = await axios.get("http://localhost:8000/users");
    console.log(response.data);
  } catch (error) {
    console.log(error);
  }
}

```

fetch	Axios
Built-in browser API	External library
Requires response.json()	Automatically handles JSON response
No installation needed	Needs installation
Manual error handling	Cleaner error handling

10. API Authentication

API authentication is the process of verifying whether a user or application is allowed to access an API. Without authentication, protected APIs could be accessed by anyone.

Authentication Method	Meaning	Common Use
API Key	A secret key used to access an API	Weather, maps, AI, payment APIs
JWT Bearer Token	Token sent after login	Protected user APIs
Basic Auth	Username and password sent in encoded form	Simple internal APIs
OAuth	Third-party login/authorization flow	Login with Google/GitHub

10.1 API Key Authentication

An API key is a secret key provided by a service to identify and authorize an application. It may be sent through headers or query parameters, depending on the API documentation.

```

fetch("https://api.example.com/data", {
  headers: {
    "x-api-key": "YOUR_API_KEY"
  }
});

```

Important security point: API keys should not be exposed in frontend code. Secret keys should be stored securely in backend environment variables.

10.2 JWT Bearer Token Authentication

JWT Bearer Token authentication is common in login-based systems. After successful login, the backend returns a token. The frontend stores this token and sends it in the Authorization header for protected API calls.

```
const token = localStorage.getItem("token");

const response = await fetch("http://localhost:8000/profile", {
  headers: {
    "Authorization": `Bearer ${token}`
  }
});

Authorization: Bearer <token>
```

10.3 OAuth Basic Idea

OAuth is used for third-party login and authorization, such as Login with Google, GitHub, or Facebook. The user does not share their password directly with our application. Instead, the third-party provider verifies the user and returns a token or user information.

11. Third-Party API Integration

Third-party API integration means connecting our application with an external service. Examples include Weather API, Google Maps API, Razorpay or Stripe payment API, OpenAI or AI model APIs, email APIs, SMS APIs, GitHub API, and News API.

Third-Party API	Use Case
Weather API	Get live weather and temperature
Google Maps API	Show maps, routes, and location data
Razorpay/Stripe API	Process online payments
AI API	Generate text, classify data, or build chatbot features
Email/SMS API	Send OTPs, alerts, and notifications
GitHub API	Fetch repository or user data

11.1 Steps for Third-Party API Integration

1. Read the API documentation carefully.
2. Get an API key or access token.
3. Understand the base URL and endpoint.
4. Check the required HTTP method.
5. Check required headers, query parameters, or request body.
6. Make the API call using fetch, Axios, or backend code.
7. Parse the JSON response.
8. Handle errors and status codes.
9. Keep secret keys secure in backend environment variables.

Best practice: For APIs that require secret keys, the frontend should call our backend, and our backend should call the third-party API. This keeps the secret key safe and gives better control over response formatting and error handling.

12. Complete Login API Integration Example

The login flow is one of the best examples of API integration because it includes frontend form handling, POST request, JSON body, backend validation, token response, and protected API access.

1. User fills login form
2. JavaScript collects email and password

3. JavaScript sends POST /login using fetch
4. Content-Type: application/json is added in headers
5. Request body is sent using JSON.stringify()
6. FastAPI receives the request
7. Pydantic validates the request body
8. Backend checks credentials
9. Backend returns JWT token or error
10. Frontend reads response using response.json()
11. Token is stored in localStorage
12. Protected API calls send Authorization: Bearer token

12.1 Frontend Login Form

```
<form id="loginForm">
  <input type="email" id="email" placeholder="Email">
  <input type="password" id="password" placeholder="Password">
  <button type="submit">Login</button>
</form>

<p id="message"></p>
```

12.2 JavaScript API Integration

```
const loginForm = document.getElementById("loginForm");

loginForm.addEventListener("submit", async function (event) {
  event.preventDefault();

  const email = document.getElementById("email").value;
  const password = document.getElementById("password").value;

  try {
    const response = await fetch("http://localhost:8000/login", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        email: email,
        password: password
      })
    });

    const data = await response.json();

    if (response.ok) {
      localStorage.setItem("token", data.access_token);
      document.getElementById("message").innerText = "Login successful";
    } else {
      document.getElementById("message").innerText = data.detail;
    }
  } catch (error) {
    document.getElementById("message").innerText = "Server not reachable";
  }
});
```

13. CORS in API Integration

CORS stands for Cross-Origin Resource Sharing. It is a browser security mechanism that controls whether a frontend from one origin can access backend APIs from another origin.

For example, if a React frontend is running on `http://localhost:3000` and a FastAPI backend is running on `http://localhost:8000`, the browser checks whether the backend allows requests from that frontend origin.

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

14. Common API Integration Mistakes

Mistake	Correct Approach
Not setting Content-Type header in POST request	Use Content-Type: application/json
Sending JavaScript object directly in body	Use JSON.stringify(object)
Forgetting to parse response	Use await response.json()
Ignoring response.ok	Check response.ok before using data as success
Exposing API keys in frontend	Store keys in backend environment variables
Not handling network errors	Use try-catch around API calls
Ignoring CORS configuration	Allow frontend origin in backend CORS settings

15. Interview Questions and Answers

Q. What is API integration?

A. API integration is the process of connecting two applications or services using APIs so they can exchange data. For example, a frontend can call a FastAPI backend to fetch products, create users, or log in users.

Q. Why is API integration important?

A. API integration is important because modern applications depend on backend servers, databases, third-party services, payment gateways, authentication providers, maps, AI APIs, and other systems.

Q. What is JSON handling?

A. JSON handling means converting JavaScript objects into JSON strings using JSON.stringify() while sending data, and converting API responses into JavaScript objects using response.json() while receiving data.

Q. How do you call an API from frontend?

A. We can call APIs from frontend using fetch or Axios. We provide the URL, HTTP method, headers, and request body if required, then read the response using response.json() or response.data in Axios.

Q. What is API authentication?

A. API authentication verifies whether a user or application is allowed to access an API. Common methods include API keys, JWT Bearer tokens, Basic Auth, and OAuth.

Q. What is a Bearer token?

A. A Bearer token is an authentication token sent in the Authorization header using the format Authorization: Bearer <token>. It is commonly used in JWT authentication.

Q. What is third-party API integration?

A. Third-party API integration means connecting our application with external services like Google Maps, Weather API, Razorpay, Stripe, email/SMS APIs, GitHub API, or AI APIs.

Q. Why should API keys not be exposed in frontend?

A. API keys are secret credentials. If exposed in frontend code, anyone can misuse them. They should be stored securely in backend environment variables.

Q. What is CORS?

A. CORS is a browser security mechanism that controls whether a frontend from one origin can access backend APIs from another origin.

Q. What is the difference between fetch and Axios?

A. fetch is a built-in browser API for HTTP requests, while Axios is an external library that simplifies API calls, automatically handles JSON responses, and provides cleaner error handling.

16. Final Summary

API integration connects applications and services through APIs. In a typical frontend-backend project, JavaScript collects user input and sends requests to backend APIs using fetch or Axios. Data is usually exchanged in JSON format. The backend validates and processes the request, communicates with databases or third-party APIs if required, and returns a JSON response with a proper status code.

For secure APIs, authentication is handled using API keys, JWT Bearer tokens, Basic Auth, or OAuth. Third-party API integration allows applications to use external services such as maps, payments, weather data, AI models, email, and SMS. Proper API integration also includes error handling, JSON handling, CORS configuration, and protection of secret keys.

Most important line: In API integration, the frontend sends requests to backend or third-party APIs using fetch or Axios, data is exchanged in JSON format, authentication is handled using API keys or JWT tokens, and proper response/error handling ensures a reliable user experience.